

Decrypt Your Data: Cryptography in Linux

Daniel Dela Dzikunu | June 8, 2026

Project Description

This lab explores practical cryptography within a Linux Bash environment. Working as user *analyst*, all files in the home directory had been encrypted, simulating a scenario where sensitive data must be recovered through cryptanalysis. The exercise required three sequential operations: reading encrypted file listings, breaking a Caesar cipher hidden inside a dotfile, and finally using OpenSSL's AES-256-CBC symmetric cipher to decrypt the main data file. Together, these tasks demonstrate how encryption secures information at rest and how the correct keys and algorithms are required to restore plaintext.

Key tools used: **ls**, **cat**, **cd**, **tr** (character translation), and **openssl**. The lab reinforces foundational concepts in symmetric encryption, key management, and file-system navigation that are central to a security analyst's daily workflow.

Task 1 — Read the Contents of a File

The first task involved listing the home directory contents and reading the README.txt file to obtain initial instructions. The **ls** command revealed three items: an encrypted file (*Q1.encrypted*), the README.txt instructions file, and a subdirectory named *caesar*. The **cat** command was then used to read README.txt, which confirmed that all data was encrypted and directed the analyst to locate a hidden file inside the caesar subdirectory in order to proceed.

Commands Used

```
ls
cat README.txt
```

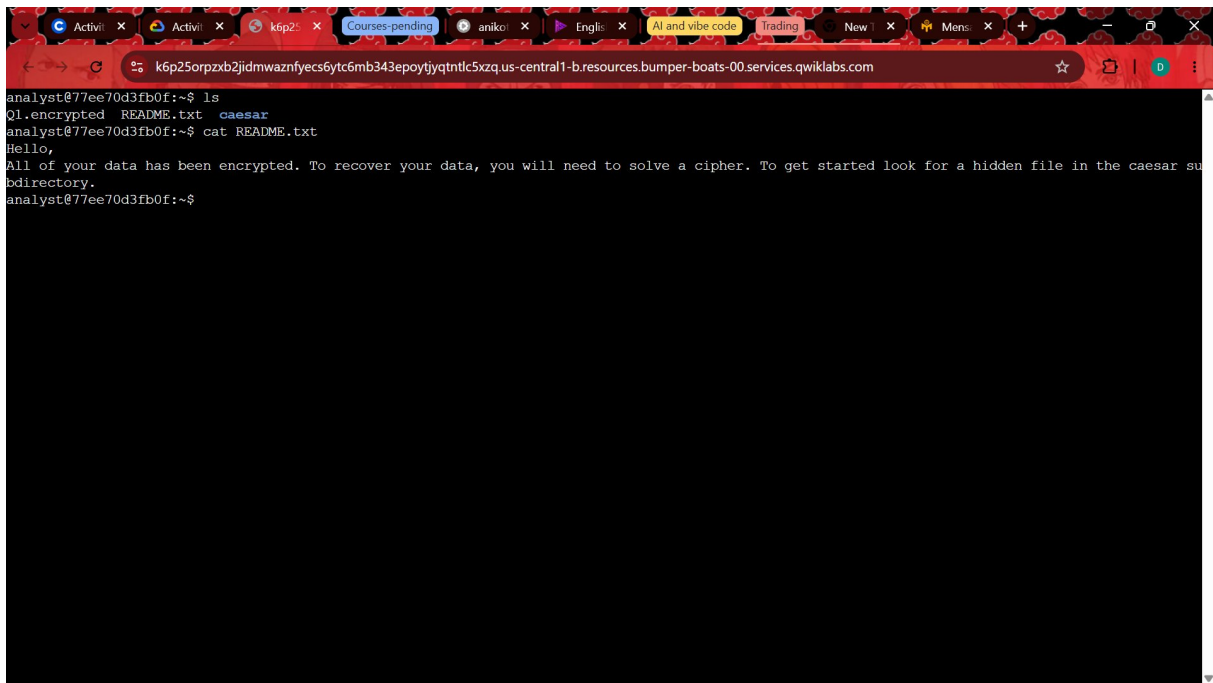


Figure 1 — `ls` output listing `Q1.encrypted`, `README.txt`, and the `caesar` directory; `cat README.txt` displaying the encrypted-data notice.

Task 2 — Find and Decrypt the Hidden File

Navigating into the `caesar` subdirectory with `cd caesar` and running `ls -a` revealed a hidden file named `.leftShift3`. In Linux, files whose names begin with a period are hidden from a standard `ls` listing; the `-a` flag makes them visible. Reading the file with `cat` produced scrambled text — the result of a Caesar cipher with a left-shift of three positions.

A Caesar cipher substitutes each letter with the letter a fixed number of positions along the alphabet. With a shift of three to the left, the letter *d* in the ciphertext represents the plaintext *a*, *e* represents *b*, and so on. The Linux `tr` command was used to reverse this substitution by mapping the shifted character set back to the standard alphabet, revealing the OpenSSL command needed for Task 3.

Locating the Hidden File

```
cd caesar
ls -a
```

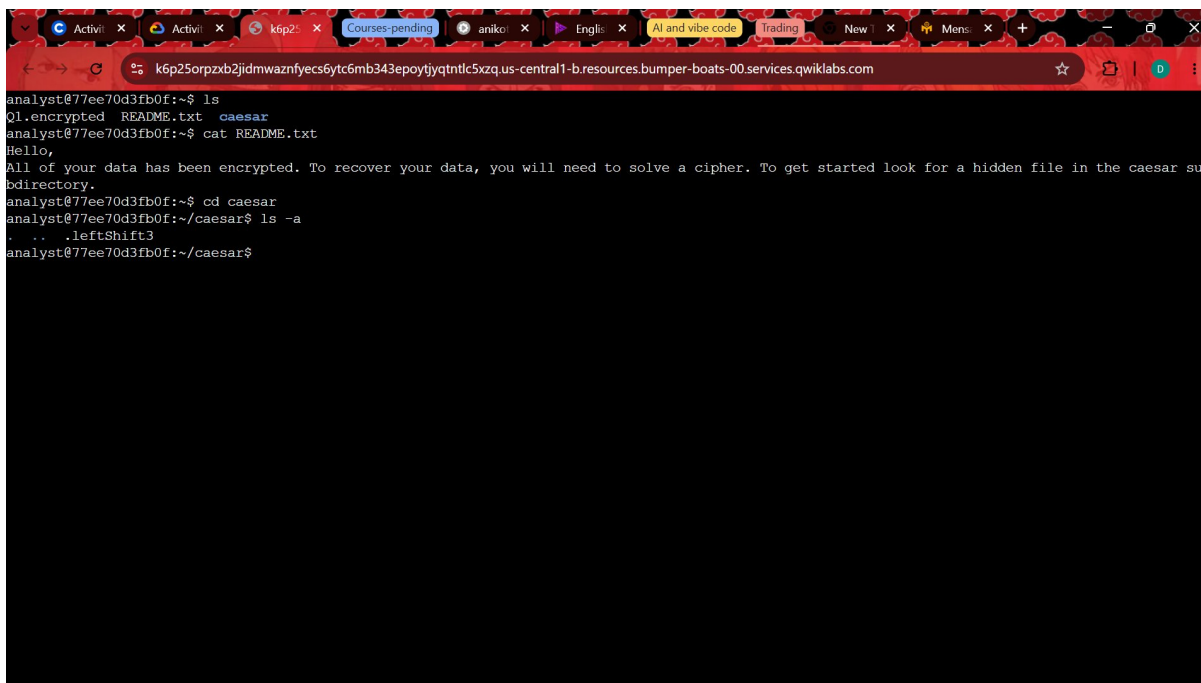


Figure 2 — `ls -la` inside the `caesar` subdirectory revealing the hidden dotfile `.leftShift3`.

Decrypting the Caesar Cipher

```
cat .leftShift3 | tr "d-za-cD-ZA-C" "a-zA-Z"
```

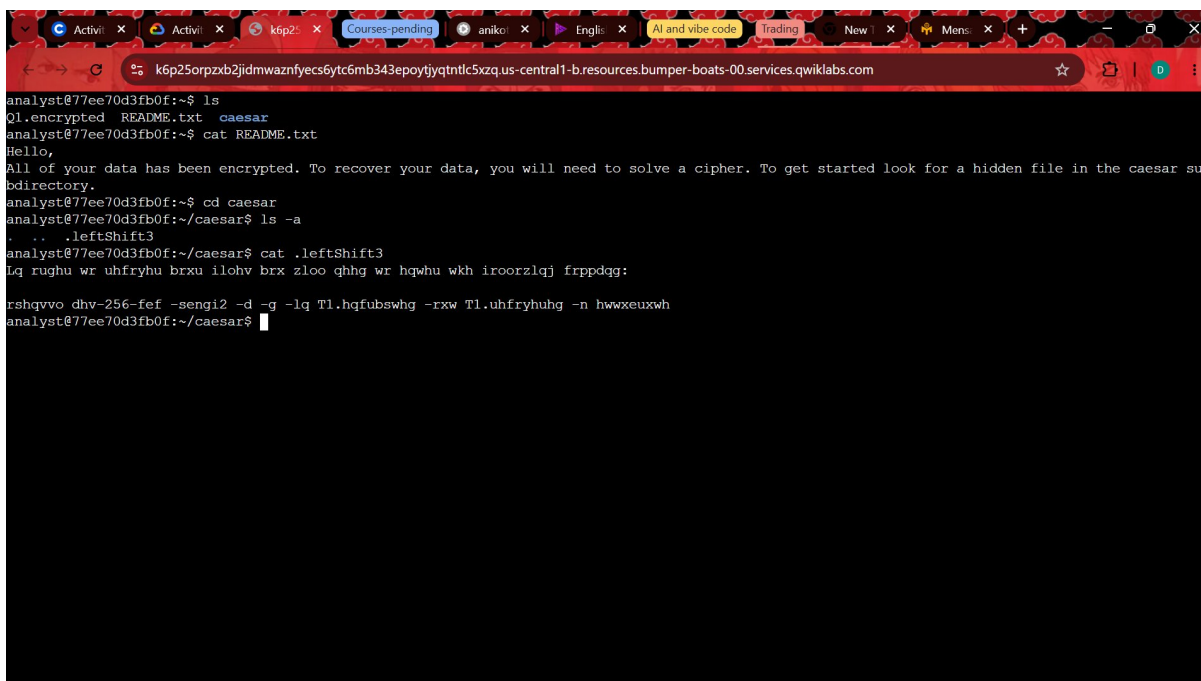


Figure 3 — `cat .leftShift3` showing ciphertext, followed by the `tr` command translating each character three positions to the right to reveal the plaintext OpenSSL decryption command.

Task 3 — Decrypt the Encrypted Data File

Returning to the home directory with `cd ~`, the decryption command decoded from Task 2 was executed. OpenSSL's **aes-256-cbc** cipher — a widely trusted symmetric encryption standard — was used to decrypt *Q1.encrypted* and write the recovered plaintext to *Q1.recovered*. The **-pbkdf2** flag applied Password-Based Key Derivation Function 2 to harden the key, **-a** handled Base64 encoding, and the password supplied via **-k** was *ettubrute* — the key revealed by the Caesar cipher.

After decryption, **ls** confirmed the presence of the new *Q1.recovered* file. Running **cat Q1.recovered** displayed the hidden message, confirming that the encryption key had been successfully recovered and the file decrypted.

Commands Used

```
cd ~
openssl aes-256-cbc -pbkdf2 -a -d -in Q1.encrypted \
    -out Q1.recovered -k ettubrute
```

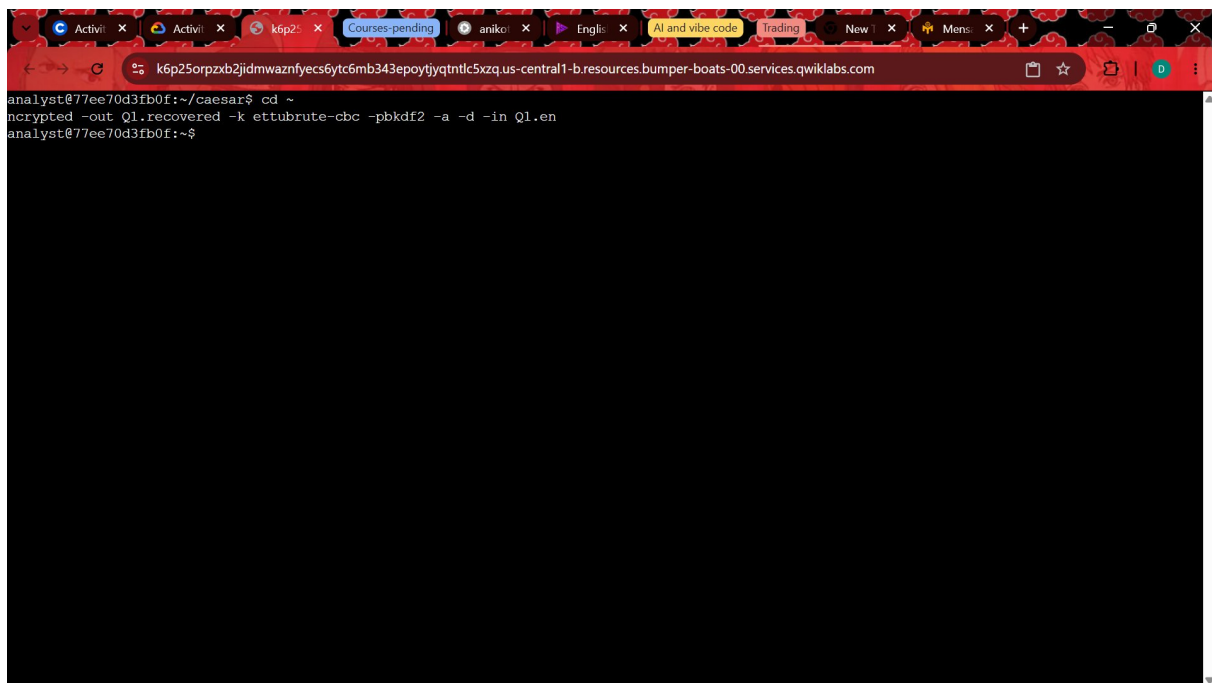
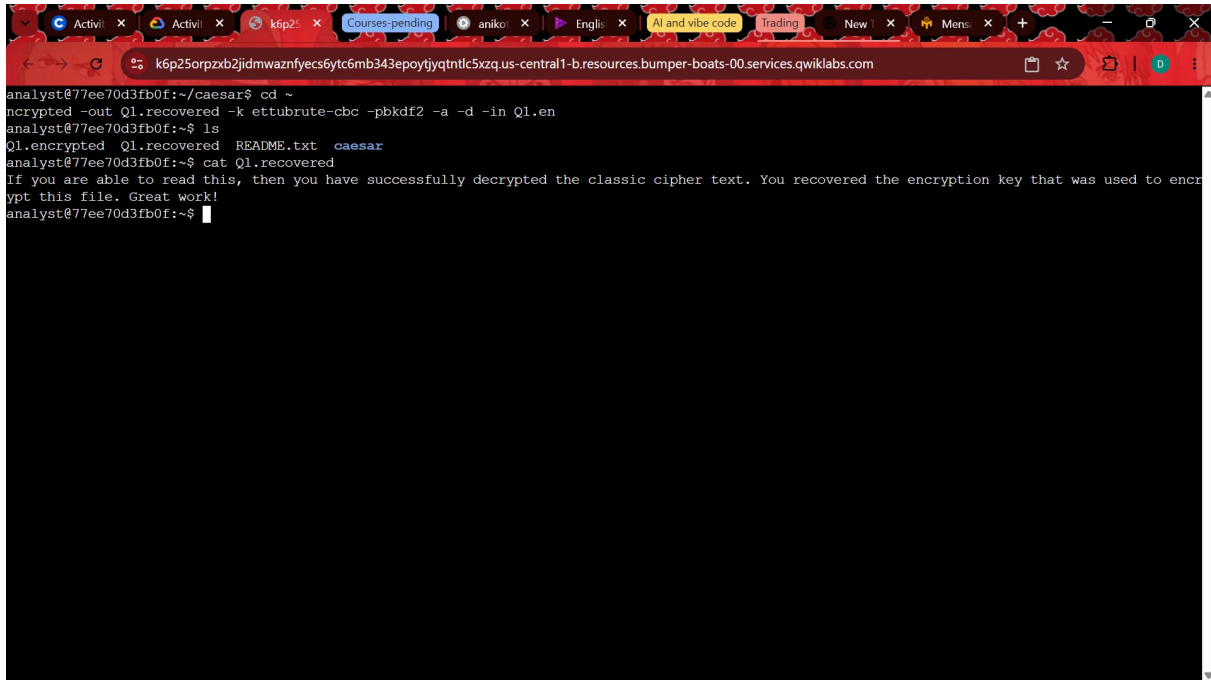


Figure 4 — Navigating back to the home directory and executing the OpenSSL AES-256-CBC decryption command with the recovered key.

Verifying the Recovered File

```
ls
cat Q1.recovered
```

A screenshot of a web browser window displaying a terminal session. The browser's address bar shows a URL from qwiklabs.com. The terminal window has a black background with white text. The user runs 'ls' and 'cat Q1.recovered', which displays a success message about decrypting a classic cipher text. The browser's tab bar at the top shows several open tabs, including 'Courses-pending', 'aniko', 'Engli', 'AI and vibe code', 'Trading', 'New', 'Mens', and others.

```
analyst@77ee70d3fb0f:~/caesar$ cd ~
analyst@77ee70d3fb0f:~$ openssl enc -d -in Q1.en -k ettubrute-cbc -pbkdf2 -a -d -in Q1.en
analyst@77ee70d3fb0f:~$ ls
Q1.encrypted Q1.recovered README.txt caesar
analyst@77ee70d3fb0f:~$ cat Q1.recovered
If you are able to read this, then you have successfully decrypted the classic cipher text. You recovered the encryption key that was used to encrypt this file. Great work!
analyst@77ee70d3fb0f:~$
```

Figure 5 — `ls` confirming `Q1.recovered` in the directory; `cat Q1.recovered` displaying the success message confirming complete decryption of the ciphertext.

Summary

This lab provided hands-on exposure to two distinct but complementary layers of cryptography. The Caesar cipher — one of the oldest known encryption techniques — demonstrated substitution-based encoding and how Linux's `tr` utility can serve as a quick decryption tool when the shift key is known. The AES-256-CBC exercise then introduced a modern symmetric encryption standard, highlighting the role of strong key derivation (PBKDF2) and proper flag usage within the OpenSSL toolkit.

Three practical skills were consolidated: listing hidden files with `ls -a`, performing character-set translation with `tr`, and decrypting files with `openssl`. From a healthcare perspective, these techniques map directly to protecting patient records — AES-256 is the cipher underpinning most healthcare data-at-rest encryption standards, and understanding how it is applied (and potentially misapplied) is essential for any analyst working at the intersection of medicine and cybersecurity.